Prompt: Build a Comprehensive Documentation System for This Repository

Analyze this entire repository and create a structured `docs/` folder that documents the project so any developer or AI agent can understand, maintain, and extend it without reverse engineering the code.

Objective

This project was built primarily using AI. I want to prevent "cognitive debt" by documenting not only what the code does, but why it exists, how everything connects, and how future features should be implemented.

The documentation should become the single source of truth for this repository.

Create this structure

```
docs/
│
├── README.md
├── PROJECT_OVERVIEW.md
├── ARCHITECTURE.md
├── FOLDER_STRUCTURE.md
├── TECH_STACK.md
├── DATABASE.md
├── API_REFERENCE.md
├── AUTHENTICATION.md
├── STATE_MANAGEMENT.md
├── ROUTES.md
├── COMPONENTS.md
├── FEATURES/
│     ├── feature-name.md
│     ├── login.md
│     ├── dashboard.md
│     ├── news.md
│     ├── admin.md
│     └── ...
├── DESIGN_DECISIONS.md
├── AI_CONTEXT.md
├── KNOWN_LIMITATIONS.md
├── SECURITY.md
├── DEPLOYMENT.md
```

```
├── TESTING.md
├── PERFORMANCE.md
├── TROUBLESHOOTING.md
├── CHANGELOG.md
└── FUTURE_IMPROVEMENTS.md
```

Documentation Requirements

For every feature, explain:

- Why it exists
- What problem it solves
- How it works internally
- Files involved
- Components involved
- APIs used
- Database tables involved
- User flow
- Admin flow
- Edge cases handled
- Validation rules
- Error handling
- Security considerations
- Performance considerations
- Future improvements

Do not simply describe the code.

Explain the reasoning behind the implementation.

Architecture Documentation

Generate diagrams (using Mermaid if possible) for:

- Folder hierarchy
- Application flow
- Authentication flow
- API request lifecycle
- Database relationships
- Component hierarchy
- User interaction flow
- Admin interaction flow

API Documentation

For every endpoint, document:

- Route
- HTTP method
- Request body
- Parameters
- Headers
- Authentication requirements
- Success response
- Error responses
- Status codes
- Validation
- Example request
- Example response

Database Documentation

Document:

- Every table
- Relationships
- Primary keys
- Foreign keys
- Indexes
- Constraints
- Purpose of each column
- How each table is used

Component Documentation

For every component explain:

- Purpose
- Props
- State
- Dependencies
- Parent components
- Child components
- Side effects

- API calls
- Rendering logic

Design Decisions

Create a document explaining:

- Why each technology was chosen
- Alternatives considered
- Trade-offs
- Important architectural decisions
- Reasons behind folder organization

AI Context

Generate an `AI_CONTEXT.md` file containing:

- Coding conventions
- Naming conventions
- Folder conventions
- Reusable utilities
- Existing patterns
- Preferred architecture
- Things an AI should never change
- Safe areas to modify
- High-risk files
- Project assumptions

This file should help future AI agents understand the project before making changes.

Feature Documentation

Create one Markdown file for every major feature.

Each feature document should contain:

- Purpose
- User workflow
- Internal workflow
- Files involved
- APIs involved

- Database usage
- Business rules
- Validation
- Error cases
- Future improvements
- Related components

Project Overview

Generate a beginner-friendly explanation covering:

- What this project does
- Main modules
- How data flows
- Authentication process
- User journey
- Admin journey
- Third-party services
- Deployment process

Troubleshooting

Document:

- Common bugs
- Causes
- Fixes
- Debugging steps
- Logs to inspect
- Frequently modified files

Documentation Quality Rules

- Do not leave placeholders.
- Analyze the real repository before writing.
- Base documentation only on existing code.
- Explain "why", not just "what".
- Use clear Markdown formatting.
- Include tables where appropriate.
- Include Mermaid diagrams whenever possible.
- Cross-reference related documentation files.

- Make the documentation useful for both humans and AI agents.

The goal is that six months from now, someone unfamiliar with this repository can understand, debug, and extend it confidently by reading the `docs/` folder without first reading the source code.